cs280_hw1

Ziyu Ge

February 2026

1 Poor Man's Augmented Reality

video result link:
https://drive.google.com/file/d/1rPQdUPw6yWLRlUUuWROqUlvTruQnDO-M/view?usp=sharing

2 Affine Transforms & Homographies

2.1

2.1.1 (b) Solving for H in `affine_solve(u,v)`

Given $u, v \in \mathbb{R}^{2\times N}$ representing N corresponding points, I restrict the affine transform to **rotation + translation only**. Thus, the transformation has the form:

$$x' = cx - sy + t_x$$
$$y' = sx + cy + t_y$$

where $c = \cos\theta$, $s = \sin\theta$, and t_x, t_y are translation parameters.
For each correspondence $(x_i, y_i) \to (x'_i, y'_i)$, I obtain two linear equations:

$$x'_i = cx_i - sy_i + t_x$$
$$y'_i = sx_i + cy_i + t_y$$

Stacking all N correspondences, I obtain a linear system:

$$A\mathbf{p} = \mathbf{b}$$

where

$$\mathbf{p} = \begin{bmatrix} c \\ s \\ t_x \\ t_y \end{bmatrix}$$

and $A \in \mathbb{R}^{2N\times 4}$, $\mathbf{b} \in \mathbb{R}^{2N}$.
Since the system may be overdetermined, solve using least squares:

$$\mathbf{p} = (A^\top A)^{-1} A^\top \mathbf{b}$$

The affine transformation matrix is then:

$$H = \begin{bmatrix} c & -s & t_x \\ s & c & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

This matrix is returned by `affine_solve(u,v)`.

2.1.2 (c) Applying the Transform in `do_transform(u,H)`

Given $u \in \mathbb{R}^{2\times N}$ and $H \in \mathbb{R}^{3\times 3}$, apply the transformation in homogeneous coordinates.

augment each point with a 1:

$$\tilde{u} = \begin{bmatrix} u \\ \mathbf{1} \end{bmatrix} \in \mathbb{R}^{3\times N}$$

$$\tilde{v} = H\tilde{u}$$

Since homogeneous coordinates are defined up to scale, divide by the third row:

$$v = \begin{bmatrix} \frac{\tilde{v}_x}{\tilde{v}_z} \\ \frac{\tilde{v}_y}{\tilde{v}_z} \end{bmatrix}$$

The resulting matrix $v \in \mathbb{R}^{2\times N}$ is returned.

2.2 solve for H in a homography

now estimate a full planar homography $H \in \mathbb{R}^{3\times 3}$ that maps points from the source image to a selected planar surface in the target image.

Using homogeneous coordinates, the mapping is defined as:

$$\tilde{v} = H\tilde{u}, \quad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}.$$

For a point (x, y) in the source image:

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1},$$
$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}.$$

The homography has 8 degrees of freedom (the scale is fixed by setting $h_{33} = 1$).

Rearranging the equations gives two linear constraints per correspondence:

$$x'(h_{31}x + h_{32}y + 1) = h_{11}x + h_{12}y + h_{13},$$
$$y'(h_{31}x + h_{32}y + 1) = h_{21}x + h_{22}y + h_{23}.$$

These can be rewritten in linear form:

$$A\mathbf{h} = \mathbf{b},$$

where

$$\mathbf{h} = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} \end{bmatrix}^\top.$$

Stacking all N correspondences yields a system $A \in \mathbb{R}^{2N\times 8}$ and $\mathbf{b} \in \mathbb{R}^{2N}$.

When $N \geq 4$, the system can be solved using least squares:

$$\mathbf{h} = (A^\top A)^{-1} A^\top \mathbf{b}.$$

The homography matrix is then reconstructed as:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}.$$

To transform all source image points, reuse the previously defined function `do_transform(u, H)`.

Each point is first converted to homogeneous coordinates:

$$\tilde{u} = \begin{bmatrix} u \\ \mathbf{1} \end{bmatrix},$$

then transformed:

$$\tilde{v} = H\tilde{u},$$

and finally converted back to Cartesian coordinates:

$$v = \begin{bmatrix} \frac{\tilde{v}_x}{\tilde{v}_z} \\ \frac{\tilde{v}_y}{\tilde{v}_z} \end{bmatrix}.$$

The transformed points are then used to construct the modified target image.

2.3

(c) Constraints on H for affine transform vs. homography For the affine transform (restricted to rotation and translation), the transformation matrix has the constrained form:

$$H_{\text{affine}} = \begin{bmatrix} c & -s & t_x \\ s & c & t_y \\ 0 & 0 & 1 \end{bmatrix},$$

where $c = \cos\theta$, $s = \sin\theta$, and t_x, t_y are translation parameters. Thus, the matrix has only 3 degrees of freedom: one rotation angle and two translations. The last row is fixed to [0 0 1], and no scaling, shear, or perspective distortion is allowed.

In contrast, a homography has the more general form:

$$H_{\text{homography}} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}.$$

Here, the matrix has 8 degrees of freedom (since it is defined up to scale). The third row is not constrained to [0 0 1], allowing projective effects. This enables perspective distortion and mapping between arbitrary planar surfaces.

(d) Can the affine transform exactly map the points? In general, the affine transform cannot exactly map the points from one image to another when the target surface involves perspective effects.

An affine transformation preserves:

- Parallelism of lines
- Ratios of distances along parallel directions

However, perspective projection does not preserve parallelism in the image plane. For example, parallel lines in 3D may converge toward a vanishing point.

Since the affine model does not include the projective terms (h_{31}, h_{32}), it cannot model this perspective distortion. Therefore, it cannot exactly align points when the surface is viewed under perspective projection.

A homography, on the other hand, includes the necessary projective terms and can exactly map points between two views of the same planar surface.

```python
import numpy as np
import cv2
import imageio.v3 as iio
import matplotlib.pyplot as plt

INPUT_VIDEO = "ar5.gif"
OUTPUT_VIDEO = "a5_out.mp4"

N_POINTS = 30
PATCH = 12
FPS = 30


# ============================================================
# 1.3 Calibrating the Camera
# - Estimate camera projection matrix P (3x4)
# - Use DLT with 2D-3D correspondences
# ============================================================

def estimate_P(Xw, uv):
    N = Xw.shape[0]
    Xh = np.hstack([Xw, np.ones((N, 1))])

    A = np.zeros((2 * N, 12), dtype=float)
    for i in range(N):
        X = Xh[i]
        u, v = uv[i]
        A[2*i,   0:4]  = X
        A[2*i,   8:12] = -u * X
        A[2*i+1, 4:8]  = X
        A[2*i+1, 8:12] = -v * X

    _, _, VT = np.linalg.svd(A)
    P = VT[-1].reshape(3, 4)
    return P


def project(P, Xw):
    Xh = np.hstack([Xw, np.ones((Xw.shape[0], 1))])
    x = (P @ Xh.T).T
    return np.stack([x[:,0]/x[:,2], x[:,1]/x[:,2]], axis=1)
```

```
# ============================================================
# 1.2 Propagating Keypoints to Other Frames
# - Initialize OpenCV trackers for each keypoint
# - Track keypoints frame-by-frame
# ============================================================

def make_tracker():
    if hasattr(cv2, "legacy") and hasattr(cv2.legacy, "TrackerMedianFlow_create"):
        return cv2.legacy.TrackerMedianFlow_create()
    if hasattr(cv2, "TrackerMedianFlow_create"):
        return cv2.TrackerMedianFlow_create()

    if hasattr(cv2, "legacy") and hasattr(cv2.legacy, "TrackerCSRT_create"):
        return cv2.legacy.TrackerCSRT_create()
    if hasattr(cv2, "TrackerCSRT_create"):
        return cv2.TrackerCSRT_create()

    if hasattr(cv2, "legacy") and hasattr(cv2.legacy, "TrackerKCF_create"):
        return cv2.legacy.TrackerKCF_create()
    if hasattr(cv2, "TrackerKCF_create"):
        return cv2.TrackerKCF_create()

    raise AttributeError("No supported OpenCV tracker found.")


def init_trackers(frame_bgr, uv0, patch=8):
    half = patch // 2
    trackers = []
    for (u, v) in uv0:
        tr = make_tracker()
        bbox = (float(u-half), float(v-half), float(patch), float(patch))
        tr.init(frame_bgr, bbox)
        trackers.append(tr)
    return trackers


def track(trackers, frame_bgr):
    uv = []
    for tr in trackers:
        ok, (x, y, w, h) = tr.update(frame_bgr)
        if not ok:
            uv.append([np.nan, np.nan])
```

```
        else:
            uv.append([x + w/2, y + h/2])
    return np.asarray(uv, dtype=float)


# ============================================================
# 1.4 Projecting a Cube into the Scene
# - Define cube in world coordinates
# - Project using calibrated P
# - Render edges on image
# ============================================================

EDGES = [(0,1),(1,2),(2,3),(3,0),
         (4,5),(5,6),(6,7),(7,4),
         (0,4),(1,5),(2,6),(3,7)]


def cube_vertices(x0, y0, z0, s):
    return np.array([
        [x0,   y0,   z0],
        [x0+s, y0,   z0],
        [x0+s, y0,   z0+s],
        [x0,   y0,   z0+s],
        [x0,   y0+s, z0],
        [x0+s, y0+s, z0],
        [x0+s, y0+s, z0+s],
        [x0,   y0+s, z0+s],
    ], dtype=float)


def draw_cube(frame_bgr, P, Vw, color=(0,0,255), thickness=2):
    uv = project(P, Vw)

    if not np.isfinite(uv).all():
        return

    uv = uv.astype(np.int32)
    for a, b in EDGES:
        cv2.line(frame_bgr,
                 (int(uv[a,0]), int(uv[a,1])),
                 (int(uv[b,0]), int(uv[b,1])),
                 color, thickness)
```

```python
# ==================== main ====================

frames = list(iio.imiter(INPUT_VIDEO))
if len(frames) == 0:
    raise RuntimeError("No frames read. Check INPUT_VIDEO.")

frame0_rgb = frames[0]
frame0_bgr = cv2.cvtColor(frame0_rgb, cv2.COLOR_RGB2BGR)


# ============================================================
# 1.1 Keypoints with Known 3D World Coordinates
# - Manually mark 2D keypoints in first frame
# - Assign structured 3D world coordinates
# ============================================================

plt.figure()
plt.imshow(frame0_rgb)
plt.title(f"Click {N_POINTS} points in THIS EXACT ORDER:\n"
          f"(1) Top face 4x5 = 20 points (front->back, left->right)\n"
          f"(2) Front face MID line = 5 points\n"
          f"(3) Front face BOTTOM line = 5 points\n"
          f"Then close window.")
uv0 = np.array(plt.ginput(n=N_POINTS, timeout=0), dtype=float)
plt.close()

H = 2.0
NX, NZ = 5, 4

Xw = []

for z in range(NZ):
    for x in range(NX):
        Xw.append([x, H, z])

for x in range(NX):
    Xw.append([x, H/2, 0])

for x in range(NX):
    Xw.append([x, 0, 0])
```

```
Xw = np.array(Xw, dtype=float)


# ============================================================
# 1.2 Initialize trackers
# ============================================================

trackers = init_trackers(frame0_bgr, uv0, patch=PATCH)


# ============================================================
# 1.4 Define cube placement
# ============================================================

x0, y0, z0, s = 1.5, H, 1.0, 1.0
Vw = cube_vertices(x0, y0, z0, s)


# ============================================================
# 1.3 Frame-by-frame calibration and projection
# ============================================================

out = []
uv_prev = uv0.copy()
P_prev = None

for fr_rgb in frames:
    fr_bgr = cv2.cvtColor(fr_rgb, cv2.COLOR_RGB2BGR)

    uv = track(trackers, fr_bgr)

    bad = np.isnan(uv[:,0]) | np.isnan(uv[:,1])
    uv[bad] = uv_prev[bad]
    uv_prev = uv

    P = estimate_P(Xw, uv)

    uv_hat = project(P, Xw)
    err = np.nanmean(np.linalg.norm(uv_hat - uv, axis=1))

    if (P_prev is not None) and (not np.isfinite(err) or err > 10.0):
        P = P_prev
```

```
    else:
        P_prev = P

    draw_cube(fr_bgr, P, Vw)
    out.append(cv2.cvtColor(fr_bgr, cv2.COLOR_BGR2RGB))


iio.imwrite(OUTPUT_VIDEO, np.stack(out), fps=FPS)
print("Wrote:", OUTPUT_VIDEO)
```

```python
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
```

```python
# Download sample images

!wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=17osoadBvt1LyZI1
!wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=1o4rgHOHhSOheowX8
!wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=15Zxo1jv_wfFkkqVl
# For the provided source image, credit to: https://www.quantamagazine.org/the-computing-pion
```

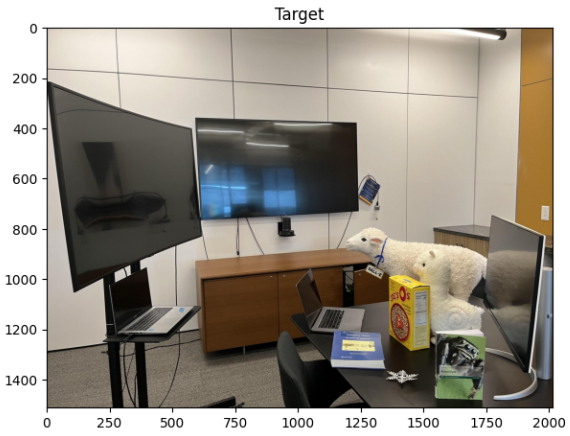
```
--2026-02-11 06:14:12--  https://docs.google.com/uc?export=download&id=17osoadBvt1LyZI1qDg_DS
Resolving docs.google.com (docs.google.com)... 142.251.2.101, 142.251.2.139, 142.251.2.113,
Connecting to docs.google.com (docs.google.com)|142.251.2.101|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=17osoadBvt1LyZI1qDg_DSrUWHQTSfIPh&
--2026-02-11 06:14:12--  https://drive.usercontent.google.com/download?id=17osoadBvt1LyZI1qDg
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 142.250.141.132, 260
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|142.250.141.132|:44
HTTP request sent, awaiting response... 200 OK
Length: 640 [application/octet-stream]
Saving to: ‘provided_corners.npy’

provided_corners.np 100%[===================>]     640  --.-KB/s    in 0s

2026-02-11 06:14:13 (22.8 MB/s) - ‘provided_corners.npy’ saved [640/640]

--2026-02-11 06:14:13--  https://docs.google.com/uc?export=download&id=1o4rgHOHhSOheowX8DX4Fu
Resolving docs.google.com (docs.google.com)... 142.251.2.101, 142.251.2.139, 142.251.2.113,
Connecting to docs.google.com (docs.google.com)|142.251.2.101|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1o4rgHOHhSOheowX8DX4Fu0k6IPAyzNzF&
--2026-02-11 06:14:13--  https://drive.usercontent.google.com/download?id=1o4rgHOHhSOheowX8D
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 142.250.141.132, 260
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|142.250.141.132|:44
HTTP request sent, awaiting response... 200 OK
Length: 155624 (152K) [image/webp]
Saving to: ‘provided_source.jpg’

provided_source.jpg 100%[===================>] 151.98K  --.-KB/s    in 0.04s
```

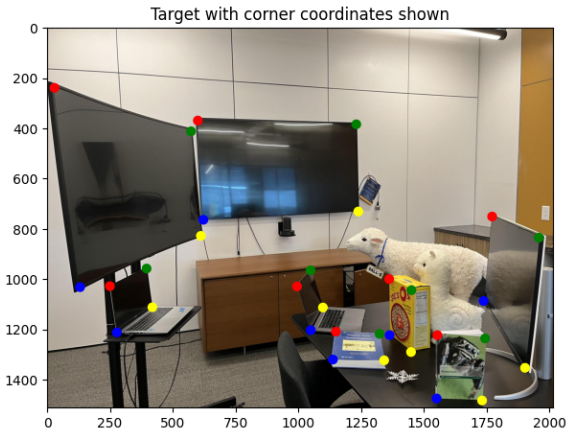
```
2026-02-11 06:14:14 (3.68 MB/s) - ‘provided_source.jpg’ saved [155624/155624]

--2026-02-11 06:14:14--  https://docs.google.com/uc?export=download&id=15Zxo1jv_wfFkkqVbZt37
Resolving docs.google.com (docs.google.com)... 142.251.2.101, 142.251.2.139, 142.251.2.113,
Connecting to docs.google.com (docs.google.com)|142.251.2.101|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=15Zxo1jv_wfFkkqVbZt37oSJJDAu86kpV
--2026-02-11 06:14:14--  https://drive.usercontent.google.com/download?id=15Zxo1jv_wfFkkqVbZ
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 142.250.141.132, 26
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|142.250.141.132|:4
HTTP request sent, awaiting response... 200 OK
Length: 318694 (311K) [image/jpeg]
Saving to: ‘provided_target.jpg’

provided_target.jpg 100%[===================>] 311.22K  --.-KB/s    in 0.06s

2026-02-11 06:14:16 (4.86 MB/s) - ‘provided_target.jpg’ saved [318694/318694]
```

```python
def reshape(x):
  """
  x: size N_locations x 4 x 2, where N_locations is the number of locations to map to in the
  Return x reshaped into 2x(4*N_locations),
  I.e., rows 0-3 in the output are the corners of the first location in the target image, ro
  """
  assert x.ndim == 3
  assert x.shape[1] == 4, x.shape[2] == 2
  y = np.zeros((2, x.shape[0]*4))

  y[0,:] = x[:,:,0].flatten()
  y[1,:] = x[:,:,1].flatten()

  return y



# Load source & target images into numpy arrays
source = np.array(Image.open('provided_source.jpg'))[:,:,:3]
target = np.array(Image.open('provided_target.jpg'))[:,:,:3]
print(f'Source shape: {source.shape}')
print(f'Target shape: {target.shape}')
```

```
# Load coordinates in target image.
# Size in file is N_locations x 4 x 2, each row is in order: top-left, top-right, bottom-left
# where N_locations is the number of locations in the target image that the source image shou
v = np.load('provided_corners.npy')
v = np.round(v).astype(int)
v = reshape(v)
print(f'v (corners) shape: {v.shape}')
print()

# Display images
plt.imshow(source)
plt.title('Source')
plt.show()
print()

fig, ax = plt.subplots(1,2, figsize=(15,10))
ax[0].set_title('Target')
ax[0].imshow(target)

ax[1].set_title('Target with corner coordinates shown')
ax[1].imshow(target)

corner_colors = ['red','green','blue','yellow']
for i in range(int(v.shape[1]/4)):
  ax[1].scatter(v[1, i*4:(i*4)+4], v[0, i*4:(i*4)+4], s=40, c=corner_colors)

plt.show()
```

```
Source shape: (968, 1720, 3)
Target shape: (1512, 2016, 3)
v (corners) shape: (2, 32)
```






```python
def affine_solve(u, v):
    # u, v: 2 x N
    N = u.shape[1]

    A = np.zeros((2*N, 6))
    b = np.zeros((2*N,))
```

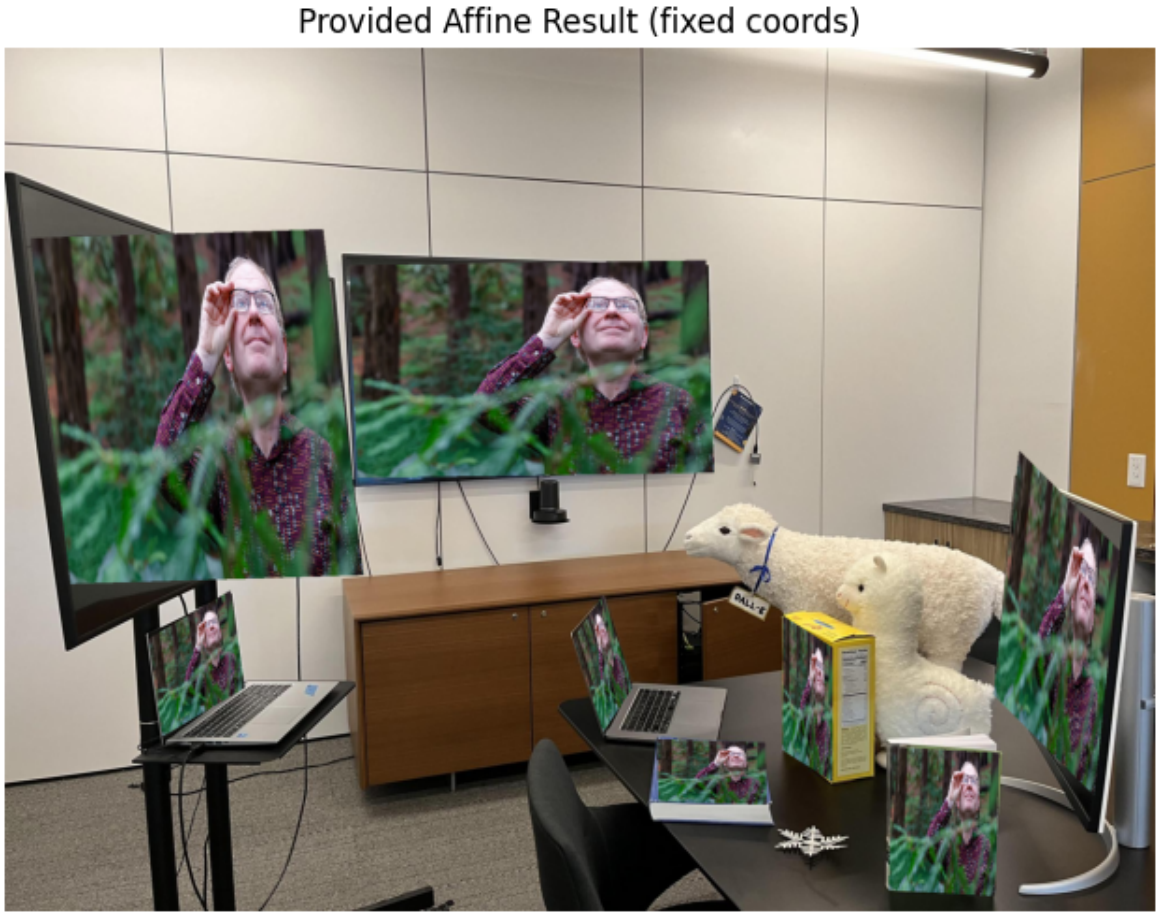
```
    for i in range(N):
        x, y = u[0, i], u[1, i]
        xp, yp = v[0, i], v[1, i]

        A[2*i]   = [x, y, 1, 0, 0, 0]
        A[2*i+1] = [0, 0, 0, x, y, 1]

        b[2*i]   = xp
        b[2*i+1] = yp

    params = np.linalg.solve(A.T @ A, A.T @ b)

    H = np.array([
        [params[0], params[1], params[2]],
        [params[3], params[4], params[5]],
        [0,         0,         1]
    ])

    return H



def homography_solve(u, v):
    # u, v: 2 x N
    N = u.shape[1]

    A = np.zeros((2*N, 8))
    b = np.zeros((2*N,))

    for i in range(N):
        x, y = u[0, i], u[1, i]
        xp, yp = v[0, i], v[1, i]

        A[2*i]   = [x, y, 1, 0, 0, 0, -x*xp, -y*xp]
        A[2*i+1] = [0, 0, 0, x, y, 1, -x*yp, -y*yp]

        b[2*i]   = xp
        b[2*i+1] = yp

    params = np.linalg.solve(A.T @ A, A.T @ b)

    H = np.array([
```

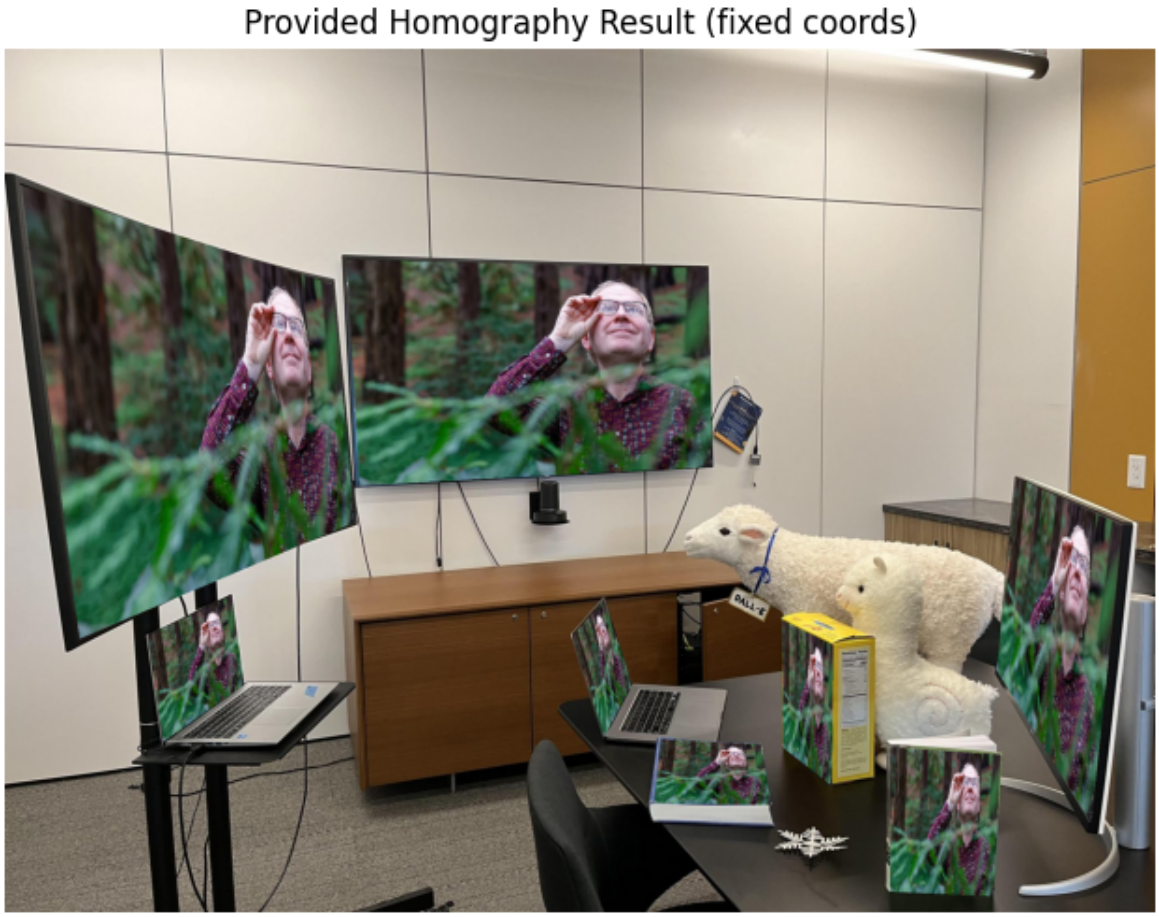
```
        [params[0], params[1], params[2]],
        [params[3], params[4], params[5]],
        [params[6], params[7], 1]
    ])

    return H



def do_transform(u, H):
    # u: 2 x N
    N = u.shape[1]

    u_h = np.vstack([u, np.ones((1, N))])  # 3 x N
    v_h = H @ u_h                          # 3 x N

    v = v_h[:2] / v_h[2]

    return v
```

```
# Perform affine & homography transforms for provided source & target pair,
# as well as a source & target pair of your choosing.

def get_source_corners(img):
    h, w = img.shape[:2]
    return np.array([
        [0,     w-1,   0,     w-1],
        [0,     0,     h-1,   h-1]
    ])

def paste_transformed(source, target, H):
    h_s, w_s = source.shape[:2]
    h_t, w_t = target.shape[:2]

    ys, xs = np.meshgrid(np.arange(h_s), np.arange(w_s), indexing='ij')
    pts = np.vstack([xs.flatten(), ys.flatten()])  # 2 x N

    pts_t = do_transform(pts, H)
    xt, yt = np.round(pts_t).astype(int)
```

```
    valid = (
        (xt >= 0) & (xt < w_t) &
        (yt >= 0) & (yt < h_t)
    )

    target_copy = target.copy()
    target_copy[yt[valid], xt[valid]] = source[
        ys.flatten()[valid], xs.flatten()[valid]
    ]

    return target_copy


# -------- Affine (provided) --------
u = get_source_corners(source)
out_affine = target.copy()

for i in range(v.shape[1] // 4):
    v_i = v[[1, 0], i*4:(i+1)*4]   # ←
    H_aff = affine_solve(u, v_i)
    out_affine = paste_transformed(source, out_affine, H_aff)

plt.figure(figsize=(10, 6))
plt.imshow(out_affine)
plt.title("Provided Affine Result (fixed coords)")
plt.axis("off")
plt.show()



# -------- Homography (provided) --------
out_homo = target.copy()

for i in range(v.shape[1] // 4):
    v_i = v[[1, 0], i*4:(i+1)*4]   # ←
    H_h = homography_solve(u, v_i)
    out_homo = paste_transformed(source, out_homo, H_h)

plt.figure(figsize=(10, 6))
plt.imshow(out_homo)
plt.title("Provided Homography Result (fixed coords)")
plt.axis("off")
```

```
plt.show()
```





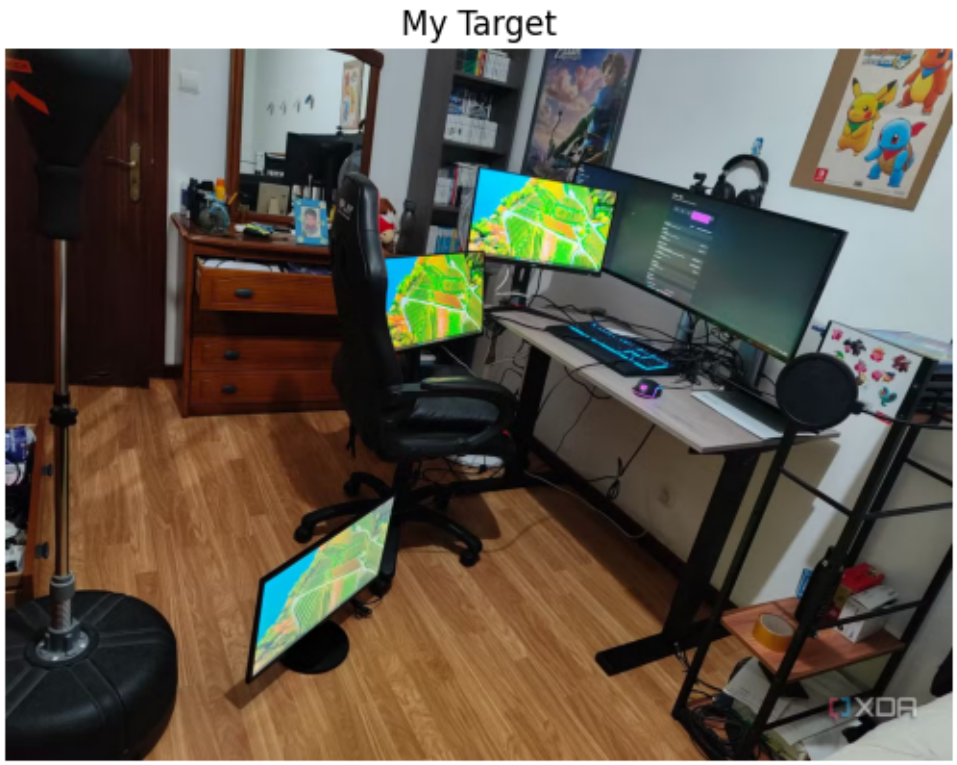

```
from google.colab import files

uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving my_source.jpeg to my_source.jpeg
Saving my_target.jpeg to my_target.jpeg
```

```
from PIL import Image
import numpy as np

my_source = np.array(Image.open("my_source.jpeg"))[:, :, :3]
my_target = np.array(Image.open("my_target.jpeg"))[:, :, :3]
```

```
plt.imshow(my_source)
plt.title("My Source")
plt.axis("off")
plt.show()

plt.imshow(my_target)
plt.title("My Target")
plt.axis("off")
plt.show()

print(my_source.shape)
print(my_target.shape)
```



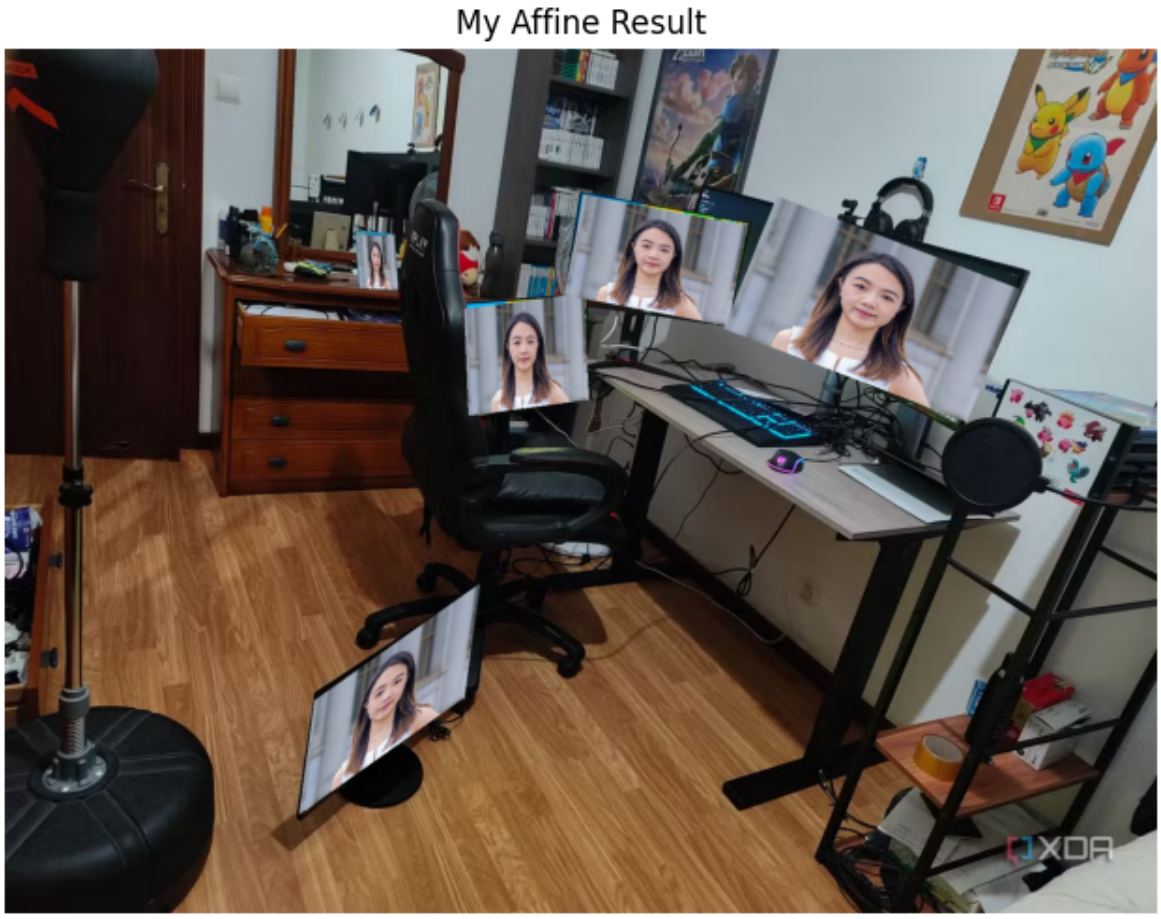

```
(680, 1028, 3)
(1238, 1650, 3)
```

```
v_my = np.array([
    [440, 683, 425, 654, 658, 829, 664, 835, 827, 1064,
     803, 1033, 1088, 1470, 1047, 1359, 504, 558, 508, 562],

    [924, 775, 1104, 919, 369, 353, 528, 498, 210, 252,
     350, 397, 225, 330, 386, 547, 266, 270, 343, 343]
])

print(v_my.shape)
```

```
(2, 20)
```

```
u_my = get_source_corners(my_source)

out_my_affine = my_target.copy()
```

```
for i in range(v_my.shape[1] // 4):
    v_i = v_my[:, i*4:(i+1)*4]
    H_aff = affine_solve(u_my, v_i)
    out_my_affine = paste_transformed(my_source, out_my_affine, H_aff)

plt.figure(figsize=(10, 6))
plt.imshow(out_my_affine)
plt.title("My Affine Result")
plt.axis("off")
plt.show()
```




```
out_my_homo = my_target.copy()

for i in range(v_my.shape[1] // 4):
```

```
    v_i = v_my[:, i*4:(i+1)*4]
    H_h = homography_solve(u_my, v_i)
    out_my_homo = paste_transformed(my_source, out_my_homo, H_h)

plt.figure(figsize=(10, 6))
plt.imshow(out_my_homo)
plt.title("My Homography Result")
plt.axis("off")
plt.show()
```